TECHNICAL PROJECT REPORT

Interactive Embedded Smart Display Prototype

ESP32-S3 • GC9A01 TFT • IMU • LittleFS • Embedded C/C++

System Block Diagram

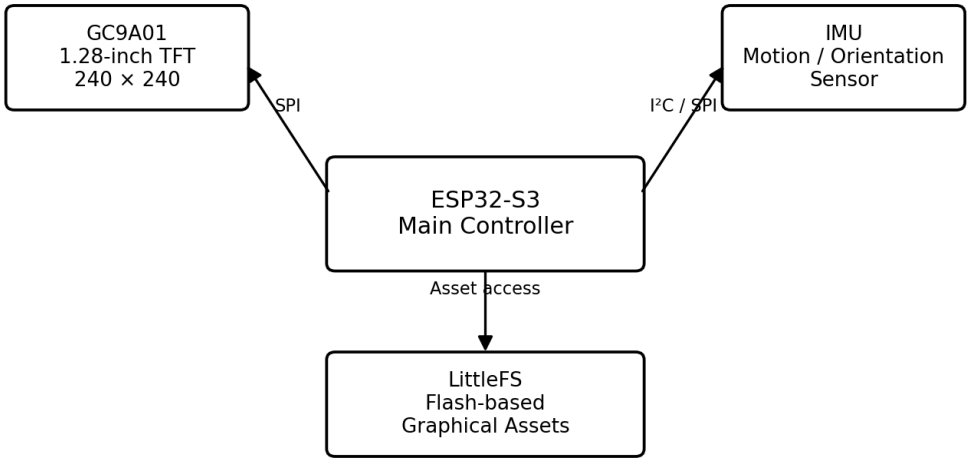


Figure 1. Overall system architecture.

Prepared by: Bhargab Gogoi

M.Tech – Smart Grid | Electrical Engineering

Contents

1. Abstract

2. Introduction

3. Problem Statement

4. Objectives

5. System Architecture

6. Hardware Components

7. Software Requirements

8. Communication Interfaces

9. Display Rendering

10. Animation System

11. IMU-Based Interaction

12. LittleFS Resource Management

13. Software Flow

14. Implementation

15. Results and Observations

16. Advantages and Limitations

17. Future Scope

18. Conclusion

19. References

1. Abstract

The Interactive Embedded Smart Display Prototype is a compact embedded system developed using an ESP32-S3 microcontroller, a GC9A01-based 1.28-inch circular TFT display, and an inertial measurement unit (IMU). The system demonstrates the integration of microcontroller programming, graphical display interfacing, sensor acquisition, animation control, and flash-based resource management.

The ESP32-S3 acts as the central processing unit and controls the graphical display while processing motion information from the IMU. The circular TFT is used to create an interactive graphical interface featuring animated facial expressions, eye movement, blinking, and wake-up sequences. Motion information can be processed to produce sensor-driven changes in displayed behavior.

Graphical resources and animation frames are managed using the ESP32-S3 flash filesystem through LittleFS. The project provides practical experience in embedded C/C++, peripheral communication, graphical rendering, sensor integration, and embedded resource management.

Keywords: ESP32-S3, Embedded Systems, GC9A01, TFT Display, IMU, LittleFS, Embedded C++, Sensor Interfacing.

2. Introduction

Embedded systems combine hardware and software to perform dedicated functions under resource and timing constraints. Modern embedded products increasingly incorporate graphical interfaces and sensors to provide more intuitive human-machine interaction.

This project focuses on developing a compact interactive display using the ESP32-S3 as the main controller. A 1.28-inch circular GC9A01 TFT provides the visual interface, while an IMU supplies motion information. Rather than displaying only static information, the prototype implements animated facial graphics including eye movement, blinking, and wake-up sequences.

The project brings together:

- Microcontroller programming and embedded C/C++
- TFT display and peripheral interfacing
- IMU sensor acquisition
- Graphical rendering and animation control
- Flash filesystem and graphical resource management
- Event-driven embedded application design

3. Problem Statement

Conventional small embedded displays are often limited to static text, numbers, or simple graphics. The objective of this project is to develop a compact graphical interface that can display animated content and respond to physical movement while remaining suitable for a resource-constrained microcontroller.

4. Objectives

- Develop an ESP32-S3-based embedded graphical system.
- Interface a GC9A01 240 × 240 circular TFT display.
- Implement animated facial graphics, eye movement, blinking, and wake-up sequences.
- Interface an IMU for motion sensing and sensor-driven interaction.
- Use LittleFS for storing and retrieving graphical assets.

- Develop a modular firmware structure for animation and peripheral management.
- Gain practical experience with embedded communication protocols and memory/resource management.

5. System Architecture

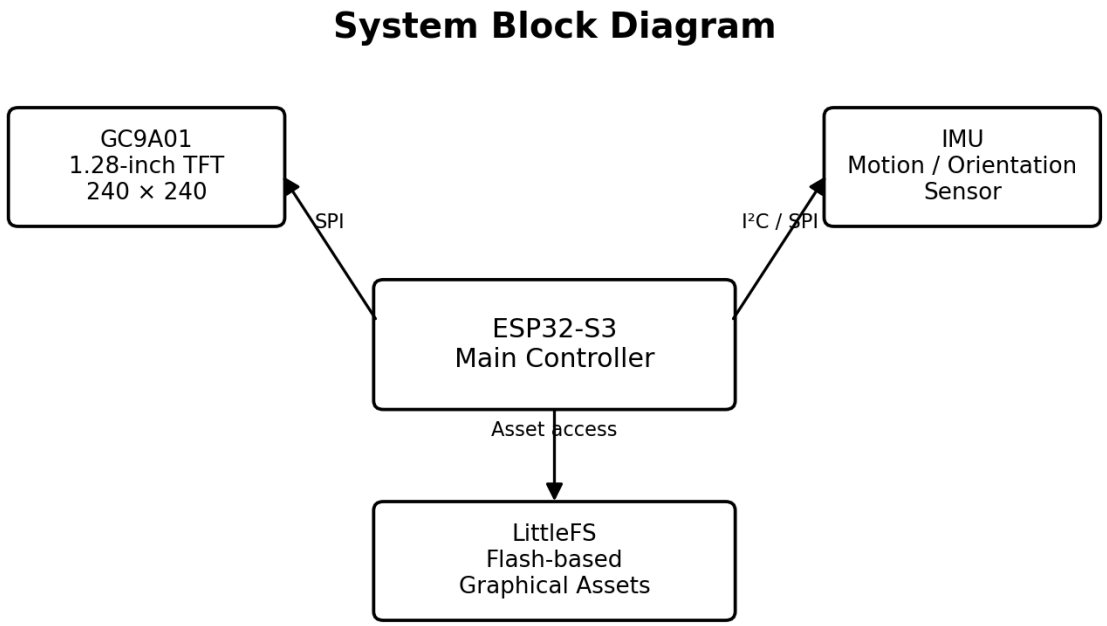


Figure 2. Functional block diagram of the prototype.

The ESP32-S3 forms the central processing unit. The TFT is used for graphical output, the IMU provides motion information, and LittleFS provides persistent storage for graphical assets. The controller coordinates these subsystems through firmware.

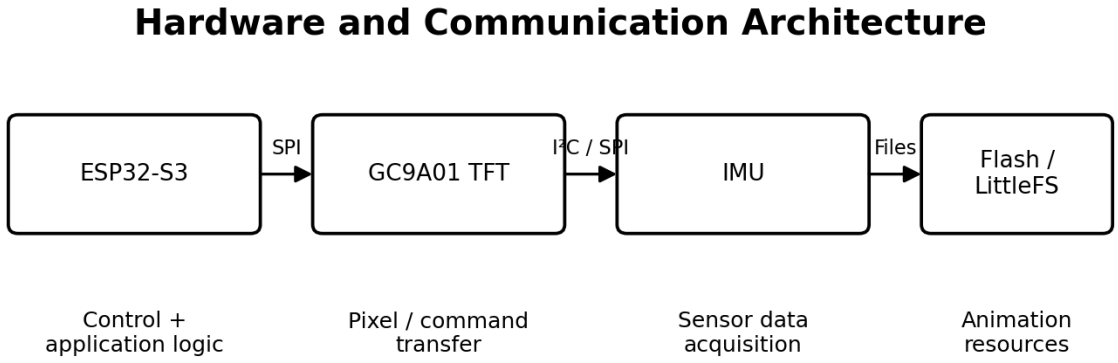


Figure 3. Hardware and communication architecture.

6. Hardware Components

6.1 ESP32-S3

The ESP32-S3 is the primary controller. The selected development configuration is based on an ESP32-S3 WROOM-1 N16R8 module, providing 16 MB flash and 8 MB PSRAM. The controller provides GPIO, SPI, I²C, wireless connectivity, and sufficient processing capability for graphical embedded applications.

6.2 GC9A01 Circular TFT

The visual interface uses a 1.28-inch GC9A01 circular TFT with a 240 × 240 pixel resolution. The circular form factor is suitable for an animated face and provides a compact graphical output device.

6.3 IMU

The IMU provides motion-related measurements such as acceleration and, depending on the selected device, angular velocity. These measurements can be used to detect movement or estimate basic orientation.

7. Software Requirements

- C/C++ embedded firmware
- Arduino IDE and/or PlatformIO
- ESP32 Arduino framework
- TFT display library/GC9A01 driver
- SPI communication for display control
- I²C or SPI communication for the IMU, depending on the selected sensor
- LittleFS for flash-based graphical asset storage

8. Communication Interfaces

8.1 SPI Display Interface

The GC9A01 display uses a serial interface, commonly configured through SPI. Typical signals include SCLK, MOSI, CS, DC, and RESET. The ESP32-S3 acts as the controller and transfers commands and pixel data to the display.

8.2 IMU Interface

The IMU may communicate with the ESP32-S3 using I²C or SPI depending on the selected sensor. In an I²C configuration, SDA and SCL carry the serial data and clock signals. The ESP32-S3 reads the sensor registers and processes the resulting measurements.

9. Display Rendering

The display resolution is 240 × 240 pixels, giving 57,600 pixels per frame. With a 16-bit RGB565 representation, a raw full frame requires approximately 115,200 bytes before accounting for additional buffers or library overhead. This makes careful frame handling and resource management important.

The animation system can update only the required graphical regions or load pre-generated frames from flash storage. This allows the ESP32-S3 to balance visual quality, memory use, and rendering performance.

10. Animation System

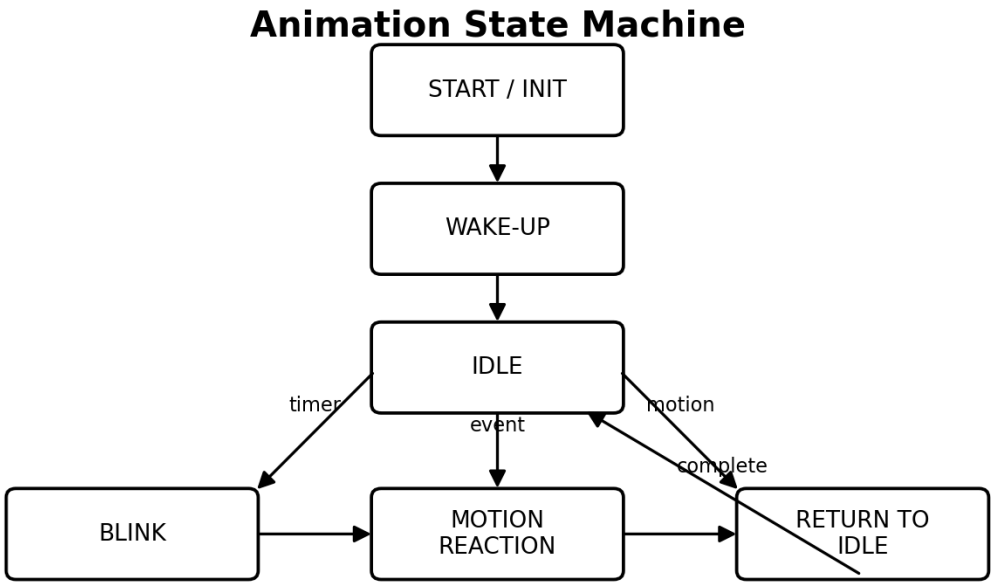


Figure 4. Example animation state machine.

The facial interface is organized into multiple animation states. A typical sequence begins with initialization and a wake-up animation, followed by an idle state. Timed events can trigger blinking, while sensor events can trigger a motion reaction. After the event is complete, the system returns to the idle state.

This state-based structure provides a clear way to coordinate sensor events, animation timing, and display updates.

11. IMU-Based Interaction

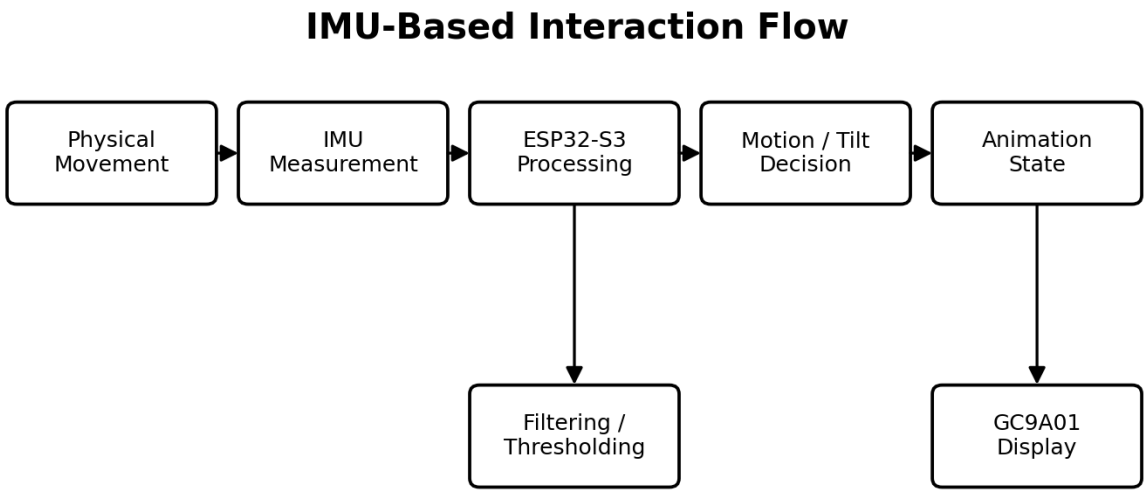


Figure 5. IMU-to-display interaction flow.

The IMU extends the system from a graphical display into an interactive embedded device. Accelerometer measurements can be used to detect movement and, under static conditions, gravity can provide a reference for basic tilt estimation. Raw measurements may be filtered or compared with thresholds before selecting an animation state.

For example, changes in the X or Y acceleration components can be mapped to left/right or up/down visual responses. More advanced versions can use calibration, low-pass filtering, complementary filtering, or sensor fusion to obtain more stable orientation estimates.

12. LittleFS Resource Management

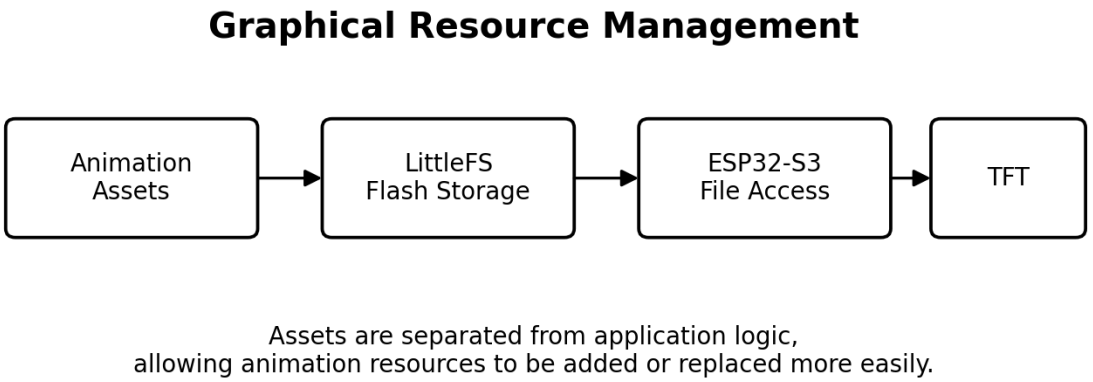


Figure 6. Graphical asset storage and retrieval path.

Graphical resources can consume significant RAM when converted into frame buffers. LittleFS provides a convenient way to keep image and animation resources in flash storage and load them when required.

A typical resource organization can separate assets into groups such as idle, blink, wake-up, and reaction frames. This keeps the application code cleaner and makes graphical resources easier to update.

13. Software Flow

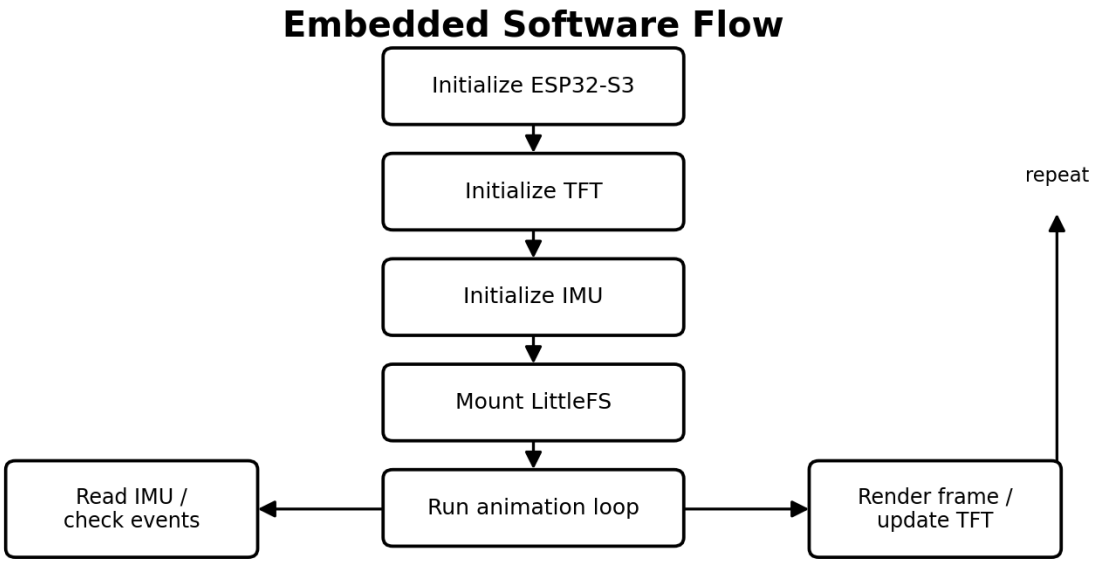


Figure 7. Simplified firmware execution flow.

At startup, the ESP32-S3 initializes its peripherals, the TFT display, the IMU interface, and LittleFS. The firmware then executes the wake-up animation and enters the main loop. During normal operation it reads sensor data, checks timing/events, determines the current animation state, renders the corresponding frame, and updates the display.

14. Implementation

Development was carried out by first establishing reliable display initialization and graphical rendering. Individual graphical elements were then combined into animation sequences. The firmware structure separates display handling, animation logic, sensor processing, and resource access so that each subsystem can be tested independently.

The IMU portion is designed around acquiring motion measurements and mapping them to interaction states. The implementation can be progressively improved by adding calibration and filtering as required. LittleFS provides persistent storage for graphical assets rather than requiring every frame to be embedded directly in the source code.

15. Results and Observations

Functionality	Outcome
ESP32-S3 control	Used as the central embedded controller
GC9A01 TFT	240 × 240 circular graphical output
Animated facial graphics	Implemented multiple visual states
Blink / eye movement	Implemented as animation sequences
Wake-up sequence	Implemented as startup animation
IMU interaction	Motion sensing and sensor-driven behavior incorporated
LittleFS	Used for flash-based graphical resource management

The prototype demonstrates practical integration of a microcontroller, graphical display, motion sensor, animation logic, and non-volatile graphical resource storage into a single embedded application.

16. Advantages and Limitations

Advantages

- Compact embedded hardware architecture.
- Graphical interface with animated visual feedback.
- Sensor-driven interaction capability.
- Modular animation and peripheral architecture.
- Flash-based asset management through LittleFS.
- ESP32-S3 provides additional wireless expansion possibilities.

Limitations

- Small display limits the amount of information that can be presented.
- Complex animation requires careful memory and rendering management.
- IMU measurements can contain noise and may require filtering.
- Basic threshold-based motion detection has limited gesture discrimination.
- A production-ready device would require further enclosure, power, reliability, and validation work.

17. Future Scope

- Advanced IMU calibration, filtering, and sensor-fusion techniques.
- Gesture recognition using multi-axis motion patterns.
- Wireless control and notification display using ESP32 connectivity.
- Voice or external AI interface integration.
- Battery-powered operation with dedicated power management.
- Custom enclosure and compact PCB integration.
- Integration with robotics and IoT systems.

18. Conclusion

The Interactive Embedded Smart Display Prototype demonstrates the integration of embedded processing, graphical display control, motion sensing, animation logic, and flash-based resource management using the ESP32-S3. The GC9A01 circular TFT provides the visual interface, while the IMU introduces sensor-driven interaction.

The project provides practical experience in embedded C/C++, SPI/I²C peripheral communication, graphical rendering, state-based software design, and memory/resource management. Its modular architecture provides a foundation for future development in robotics, IoT, interactive devices, and other embedded human-machine interfaces.

19. References

- Espressif Systems, ESP32-S3 Series Datasheet.
- Espressif Systems, ESP32-S3 Technical Reference Manual.
- Sitronix, GC9A01 LCD Controller Datasheet.
- Espressif Systems, Arduino-ESP32 Documentation.
- Arduino Documentation, Arduino Programming Reference.
- TFT_eSPI Library Documentation.
- LittleFS filesystem documentation.
- Technical documentation and datasheet of the selected IMU sensor.